

Minor Project Progress Report

ARP-455 | 1st Mid-Term Evaluation

Oct 8, 2026 · @Satyam

Submitted by: Satyam Chhetri & Aditya Dhankar

Faculty Supervisor: Prof. Anupam Lakhanpal

Department of Computer Science & Engineering

Academic Year 2026–2027

1. Title of the Project

Multimodal AI Chatbot with Retrieval-Augmented Generation (RAG) for Smart Campus Assistance

2. Introduction

Educational institutions generate and maintain a vast volume of information — academic regulations, examination schedules, fee structures, course catalogues, departmental notices, facility details, event announcements and student-support policies. This information is typically scattered across PDFs, notice boards, departmental websites and circulars, forcing students to spend considerable time and effort locating what they need. Conventional keyword-based search or static FAQ pages fail to understand natural, conversational questions and cannot keep pace with frequently changing campus documents.

This project proposes a Multimodal AI-based Smart Campus Assistant that allows students, faculty and administrative staff to interact with a chatbot using both natural-language text and images (such as photographed notices or posters). The system is built around Retrieval-Augmented Generation (RAG), a technique that grounds a Large Language Model's (LLM) response in factual content retrieved from an authoritative knowledge base — the institution's own documents — rather than relying purely on the model's parametric memory.

The proposed assistant is engineered as a full-stack web application. The client experience is delivered through a React.js single-page application (the front-end tier of the MERN stack), while MongoDB serves as the primary operational database. The AI/RAG orchestration layer is implemented in Django (Python). Semantic search over document embeddings is powered by Pinecone, a cloud-native vector database.

Progress to Date — Month 1:

For the first month, the team has focused entirely on the Admin Module — the core knowledge-base management layer. The admin can log in securely (JWT-based authentication), upload college notices in PDF or plain-text format, which are automatically parsed, chunked, embedded using Sentence-Transformers, and indexed into Pinecone. The admin dashboard lists all uploaded documents with their indexing status and provides a test-retrieval interface that demonstrates the working RAG pipeline: a query typed by the admin returns the top-3 semantically matching chunks from the knowledge base, confirming that the retrieval foundation is functioning correctly before the student-facing chat UI is built.

3. Literature Survey

3.1 Large Language Models (LLMs)

Transformer-based Large Language Models such as GPT, Gemini and LLaMA have demonstrated strong capabilities in natural-language understanding, reasoning and text generation. However, these models are trained on static corpora with a fixed knowledge cut-off and are prone to "hallucination" — generating plausible-sounding but factually incorrect statements — particularly when queried on information outside their training data, such as institutional policies or recent notices.

3.2 Retrieval-Augmented Generation (RAG)

RAG, first formalised by Lewis et al. (2020), addresses this limitation by coupling an LLM with an external, searchable knowledge base. At query time, the most semantically relevant passages are retrieved and injected into the model's prompt as context, so the generated answer is grounded in verifiable source material. This significantly reduces hallucination, allows the knowledge base to be updated without retraining the model, and enables the system to cite the exact source of an answer.

3.3 Vector Databases and Semantic Search

Traditional keyword search fails when a student phrases a question differently from the wording in a document. Embedding models convert text into dense numeric vectors that capture semantic meaning, and a vector database indexes these for fast approximate-nearest-neighbour search. Managed vector databases such as Pinecone abstract away the operational complexity of index management and scaling.

3.4 Multimodal AI

Vision-language models extend text-only LLMs by allowing images to be interpreted alongside text prompts. In a campus context, this enables the assistant to read a photographed notice, timetable or poster and answer questions about its contents.

3.5 AI Chatbots in Education

Prior educational chatbot deployments have generally relied on rule-based decision trees or static FAQ databases. Such systems handle only a narrow set of anticipated questions and require manual reprogramming whenever institutional information changes. They also typically lack any mechanism to substantiate an answer with a source document, reducing user trust.

3.6 Research Gap

Existing campus chatbot systems are largely FAQ-driven, cannot dynamically incorporate newly published documents, and rarely support multimodal input. The proposed system addresses this gap by combining RAG-based dynamic retrieval, multimodal (text + image) interaction, and a modern full-stack architecture (React, Django, MongoDB, Pinecone) tailored specifically to campus information needs.

4. Problem Statement

Students frequently face difficulty obtaining accurate, up-to-date campus information because it is fragmented across notices, PDFs, departmental websites and physical circulars. Traditional FAQ-based or rule-based chatbots cannot interpret complex or conversationally phrased questions, cannot dynamically retrieve information from newly added documents, and offer no way to verify where an answer originated.

There is a clear need for an intelligent campus assistant capable of:

- Understanding natural-language questions posed in everyday phrasing.
- Dynamically retrieving relevant information from an evolving set of college documents.
- Generating accurate, context-aware answers grounded in verified institutional content.

- Processing both textual and image-based (e.g., photographed notice) queries.
- Providing transparent source references so students can verify an answer's origin.

5. Motivation

The motivation for this project stems from the everyday friction students experience when trying to locate accurate campus information, and from the opportunity that modern generative-AI and retrieval techniques present to solve this problem robustly:

- Reduce the time students spend searching across multiple notices, PDFs and websites.
- Provide round-the-clock (24×7) automated assistance without waiting on office hours.
- Reduce repetitive query handling workload on administrative and faculty staff.
- Make institutional information more discoverable through natural conversation.
- Improve the factual accuracy of AI-generated responses through retrieval grounding (RAG).
- Demonstrate a practical, deployable full-stack AI system combining modern web and ML technologies.

6. Proposed Design Flow

System Workflow (End-to-End):

```

Student (Text or Image Query)
    ↓
React.js Chat Interface
    ↓
Express.js API Gateway (Auth & Routing)
    ↓
Django RAG Service – Query Embedding
    ↓
Pinecone Vector Search (Top-k Retrieval)
    ↓
Context-Augmented Prompt Construction
    ↓
LLM (OpenAI / Gemini) – Answer Generation
    ↓
Response + Source Citations
    ↓
MongoDB – Conversation Persisted
    ↓
Answer Displayed to Student

```

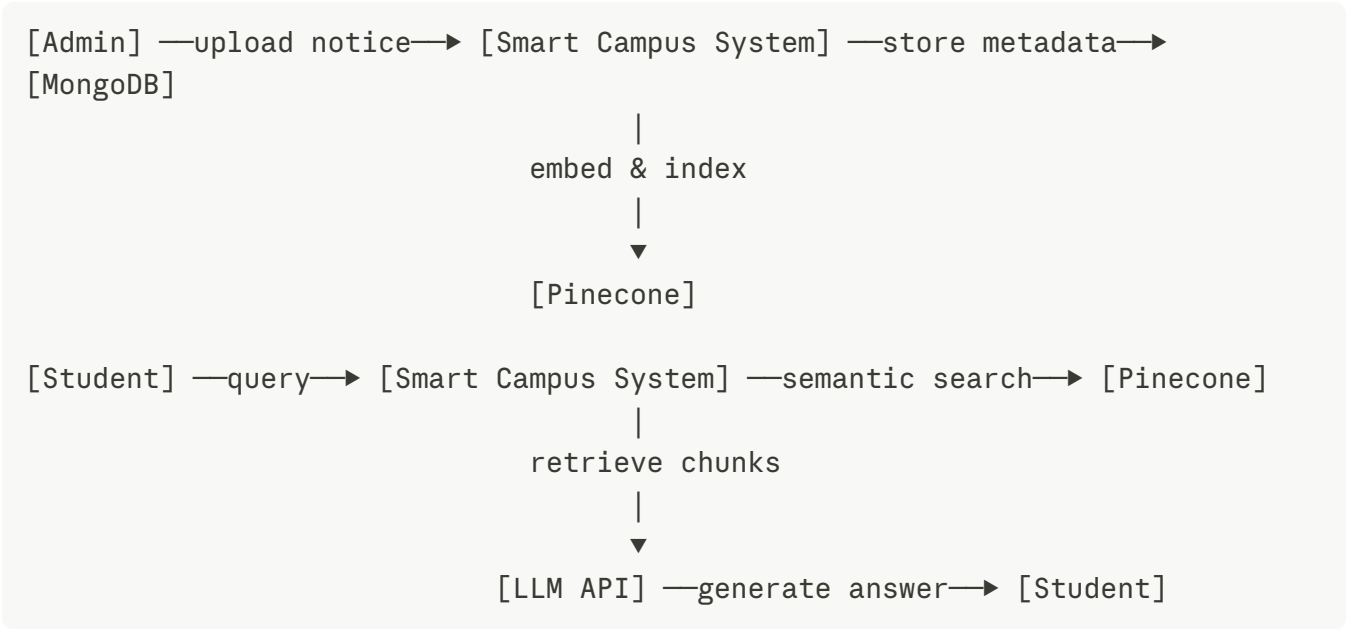
Admin Knowledge-Base Flow (Month 1 — Implemented):

```

Admin Login (JWT Auth)
    ↓
Upload Notice (PDF / Text) via React Admin Panel
    ↓
Express API → MongoDB (document metadata saved, status: pending)
    ↓
Django Ingest Endpoint called
    ↓
Text Extraction (pdfplumber) → Chunking → Embedding (Sentence-Transformers)
    ↓
Pinecone Upsert (vectors + metadata: doc_id, page, source)
    ↓
MongoDB document status updated → "indexed"
    ↓
Admin Dashboard: lists documents with status, chunk count, upload date
    ↓
Test-Retrieval Box: admin types query → top-3 matching chunks returned

```

Data Flow Diagram (Level 0):



ER Diagram (Key Entities):

Entity	Key Attributes	Relationship
USERS	_id, name, email, role, password	1 user → N conversations
CONVERSATIONS	_id, user_id (FK), created_at, title	1 conversation → N messages
MESSAGES	_id, conversation_id, role, content, timestamp	Belongs to conversation
DOCUMENTS	_id, title, department, category, file_path, status, chunk_count, upload_date	1 document → N Pinecone vectors
PINECONE INDEX	vector_id, embedding, doc_id (FK), chunk_text, source, page_number	Referenced by DOCUMENTS

7. Proposed Methodology

The project follows an iterative, milestone-based development approach across a 10-week schedule:

Month 1 — Weeks 1-4 (Admin Module & RAG Pipeline Foundation) — Current

1. Requirement analysis and tech stack setup: React.js, Node.js/Express.js, Django, MongoDB, Pinecone.
2. Admin authentication module with JWT-based login and role-based access control (admin / student).
3. Admin panel (React.js) for uploading college notices in PDF or plain-text format.
4. Express.js API to save document metadata to MongoDB (`status: pending`) and trigger the Django ingest service.
5. Django ingest endpoint: PDF parsing (pdfplumber) → text chunking → embedding (Sentence-Transformers) → Pinecone upsert with metadata (doc_id, page, source).
6. Admin dashboard: lists all documents with status (pending / indexed / failed), chunk count and upload date, plus a delete button that also removes the vectors from Pinecone.
7. Test-retrieval interface on the dashboard: admin enters a query and receives the top-3 semantically matching chunks — demonstrating the core RAG retrieval pipeline without LLM involvement.

Month 2 — Weeks 5-7 (Student Chat Interface & LLM Integration) — Planned

- Student-facing chat UI in React.js with multi-turn conversation history.
- LLM integration (OpenAI / Gemini API) to generate natural-language answers from retrieved chunks.
- Source citation displayed alongside every generated answer for transparency.

Month 3 — Weeks 8-10 (Multimodal, Testing & Deployment) — Planned

- Image upload and vision-model processing for photographed notices and posters.
- End-to-end integration testing, retrieval accuracy evaluation and performance tuning.
- Final documentation and project demonstration.

8. Results (if any)

The following components are functional at the end of Month 1:

Component	Status
Admin Login (JWT Auth)	✔ Complete
Admin — Upload Notice (PDF / Text)	✔ Complete

Component	Status
Express API — MongoDB Document Record	✔ Complete
Django Ingest Service (Parse → Chunk → Embed → Pinecone)	✔ Complete
Admin Dashboard (Document List + Status)	✔ Complete
Test-Retrieval Interface (Semantic Search Demo)	✔ Complete
Student Chat UI	☐ Planned (Month 2)
LLM Answer Generation	☐ Planned (Month 2)
Multimodal / Image Support	☐ Planned (Month 3)

Key Result: A notice uploaded through the React admin panel is automatically parsed, chunked, embedded and indexed into Pinecone within seconds. When a semantic query is typed into the test-retrieval box (e.g., "when is the fee payment deadline?"), the correct paragraph from the uploaded notice is returned as the top match — confirming that the retrieval foundation of the RAG pipeline is working correctly.

9. GANTT / PERT Chart

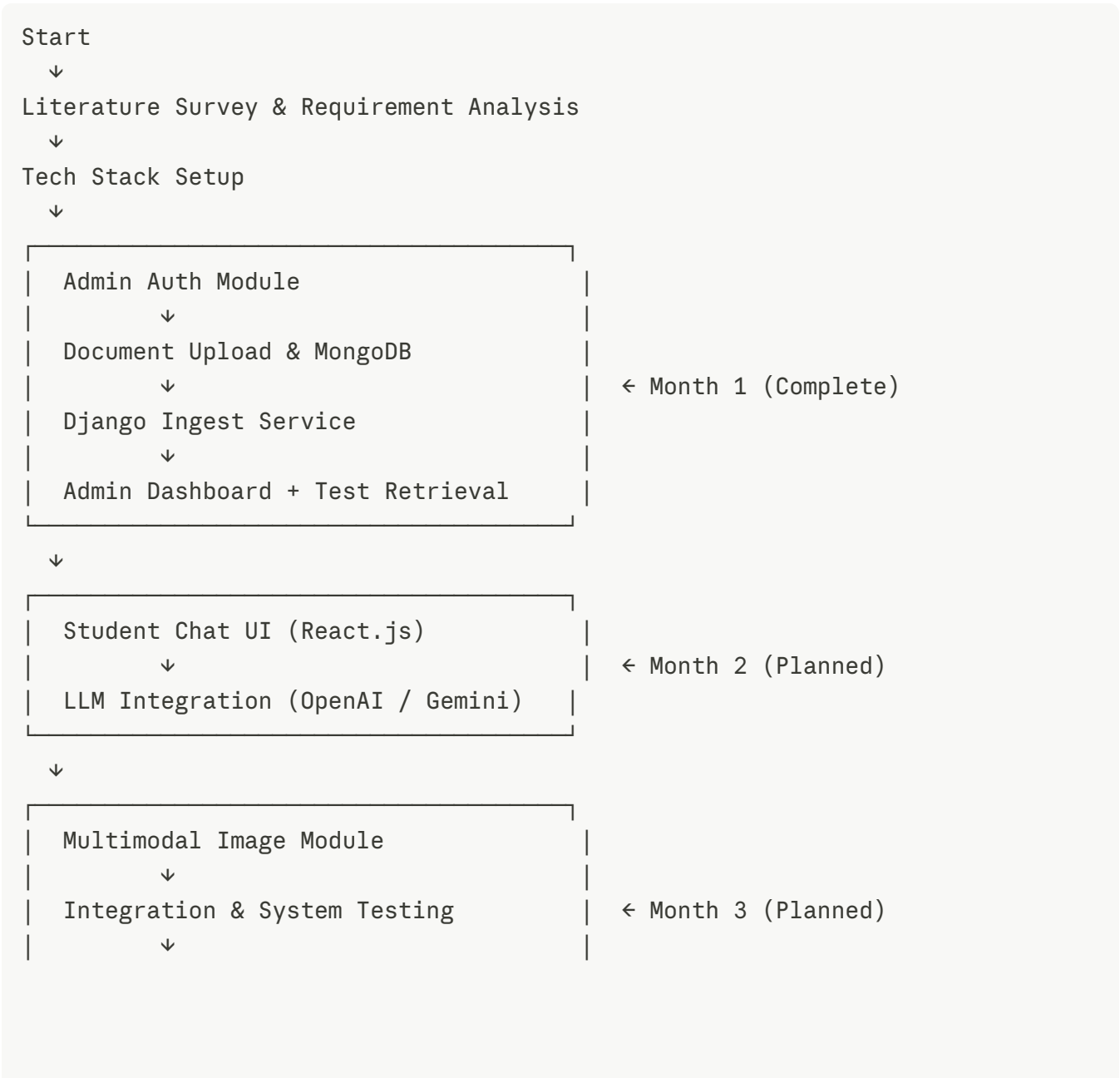
Gantt Chart:

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10
Literature Survey & Requirement Analysis	■	■								
Tech Stack Setup (MERN, Django, MongoDB, Pinecone)	■	■								
Admin Auth Module (JWT Login)		■	■							
Document Upload & MongoDB Integration			■	■						
Django Ingest Service (Parse, Chunk, Embed, Pinecone)			■	■						
Admin Dashboard & Test-Retrieval Interface				■						
Student Chat UI (React.js)					■	■				

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10
LLM Integration (OpenAI / Gemini)						■	■			
Multimodal / Image Module							■	■		
Integration & System Testing								■	■	
Documentation & Final Presentation									■	■

(■ = Active / Completed; current progress through end of Week 4)

PERT Chart (Activity Dependencies):



Documentation & Final Presentation



End

10. Hardware and Software Requirements

Hardware Requirements (Development Environment):

Component	Specification
Processor	Intel Core i5 (10th Gen) or equivalent, 2.4 GHz+
RAM	8 GB minimum, 16 GB recommended
Storage	20 GB available SSD space
Display	1366 × 768 or higher resolution
Network	Stable broadband internet connection

Since AI/LLM inference, embedding generation and vector search are performed through cloud-based APIs (OpenAI/Gemini and Pinecone), a dedicated local GPU is not required.

Software Requirements:

Category	Technology / Tool
Operating System	Windows 10 / 11 (development); Linux (production server)
Frontend	React.js, HTML5, CSS3, Tailwind CSS
API Gateway	Node.js, Express.js
AI / Backend Service	Python 3.x, Django, Django REST Framework
Primary Database	MongoDB (MongoDB Atlas)
Vector Database	Pinecone (managed cloud vector store)
Embedding Model	Sentence-Transformers
LLM / Vision API	OpenAI API / Google Gemini API

Category	Technology / Tool
RAG Orchestration	LangChain (Python)
PDF Processing	pdfplumber
Version Control	Git & GitHub
Development Tools	VS Code, Postman, MongoDB Compass, Pinecone Console
Deployment (optional)	Docker, Render / AWS / Vercel

11. References

1. Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems (NeurIPS)*.
2. Reimers, N., & Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *Proceedings of EMNLP*.
3. Pinecone Systems Inc. — Official Documentation. <https://docs.pinecone.io>
4. LangChain — Official Documentation. <https://python.langchain.com>
5. Django Software Foundation — Django REST Framework Documentation. <https://www.django-rest-framework.org>
6. MongoDB Inc. — Official Documentation. <https://www.mongodb.com/docs>
7. OpenAI Platform Documentation. <https://platform.openai.com/docs>
8. Google AI — Gemini API Documentation. <https://ai.google.dev>
9. React — Official Documentation. <https://react.dev>